UNIVERSIDAD CATÓLICA SAN PABLO (UCSP) · FORMACIÓN DE EMPRESAS DE BASE TECNOLÓGICA I

PichangaBook Arequipa — Informe Técnico de Avance MVP

Demostración ejecutable de Frontend Móvil/Web, Backend NestJS, Base de Datos PostgreSQL (ACID) e Integraciones en Tiempo Real

Entrega: Presentación Avance MVP (28 oct 2026)
Integrantes: Rodrigo Sierra · Samuel Iman
Ciudad foco: Arequipa, Perú (UTC-5)

1. Stack Tecnológico Comprometido e Implementado en el Monorepo

Capa / Módulo	Tecnología Comprometida	Implementación Real en el Código
Frontend Móvil (Jugadores / Org.)	**Flutter (Dart)** + Espejo Web Interactivo	`/mobile-flutter/lib/main.dart` + 4 pantallas Dart y `demo-app-mobile.html` conectado en vivo.
Frontend Web (Panel Dueños)	**HTML5, CSS3, Tailwind CSS**	`/web-admin-tailwind/index.html` con agenda en vivo, KPIs, bloqueo en recepción y subida de RUC.
Backend Core	**Node.js + NestJS (TypeScript)**	`/backend-nestjs/src/` con módulos `auth`, `complexes`, `bookings`, `substitutes`, `integrations`.
Base de Datos	**PostgreSQL** (Transacciones ACID)	`schema.sql` con 10 tablas, `EXCLUDE USING gist` anti doble-booking y `phone_masked`.
Tiempo Real	**WebSockets (Socket.io)**	`BookingsGateway` bidireccional (`slot:updated`, `split:updated`, `slot:block`) en < 50 ms.
Geolocalización	**Google Maps API** (Maps, Places, Geocoding)	`MapsService` + pines GPS en Cayma, Yanahuara, Cerro Colorado, J.L.B. y Rivero y Sachaca.
Multimedia	**Cloudinary / AWS S3** (Clips 15-30s)	`MediaService.signClipUpload` (SHA-1, 480p, H.264) + 4 clips MP4 de 18s a 28s operativos.
Pasarela Pagos	**Culqi / Mercado Pago** (Yape, Plin, Tarjeta)	`CulqiService` + webhook firmado `HMAC-SHA256` e idempotencia por `provider_charge_id`.
Seguridad Auth	**Firebase / Supabase Auth** (OTP SMS)	`OtpService` + `AuthGuard` JWT por rol + enmascaramiento y proxy VoIP contra extorsión.
Infraestructura	**Docker + Render / AWS** (HTTPS/SSL)	`docker-compose.yml` (PostgreSQL 16 + NestJS), `Dockerfile` multi-stage y `render.yaml`.

2. Estructura Física del Monorepo Entregado

```
PichangaBook v.1/
├── docker-compose.yml              # PostgreSQL 16 + API NestJS
├── render.yaml                     # Cloud Hosting + SSL/TLS
├── backend-nestjs/                 # Servidor REST + Socket.io
│   ├── schema.sql & seed.sql       # Esquema ACID + Semilla AQP
│   ├── public/media/clips/         # 4 clips MP4 (18s-28s) + fotos
│   └── src/
│       ├── auth/                   # OTP SMS + Roles + JWT Guard
│       ├── complexes/              # Mapa AQP + RUC SUNAT + Grilla
│       ├── bookings/               # ACID + Split + BookingsGateway
│       ├── substitutes/            # Bolsa Suplentes + Chat/VoIP
│       └── integrations/           # Maps, Cloudinary/S3, Culqi
├── web-admin-tailwind/index.html   # Dashboard Web Dueños/Recepción
└── mobile-flutter/                 # App Móvil Jugador/Organizador
    ├── pubspec.yaml & lib/         # Código nativo Flutter (Dart)
    └── demo-app-mobile.html        # Espejo móvil conectado en vivo
```

3. Evidencia de Verificación E2E y Concurrencia en Consola

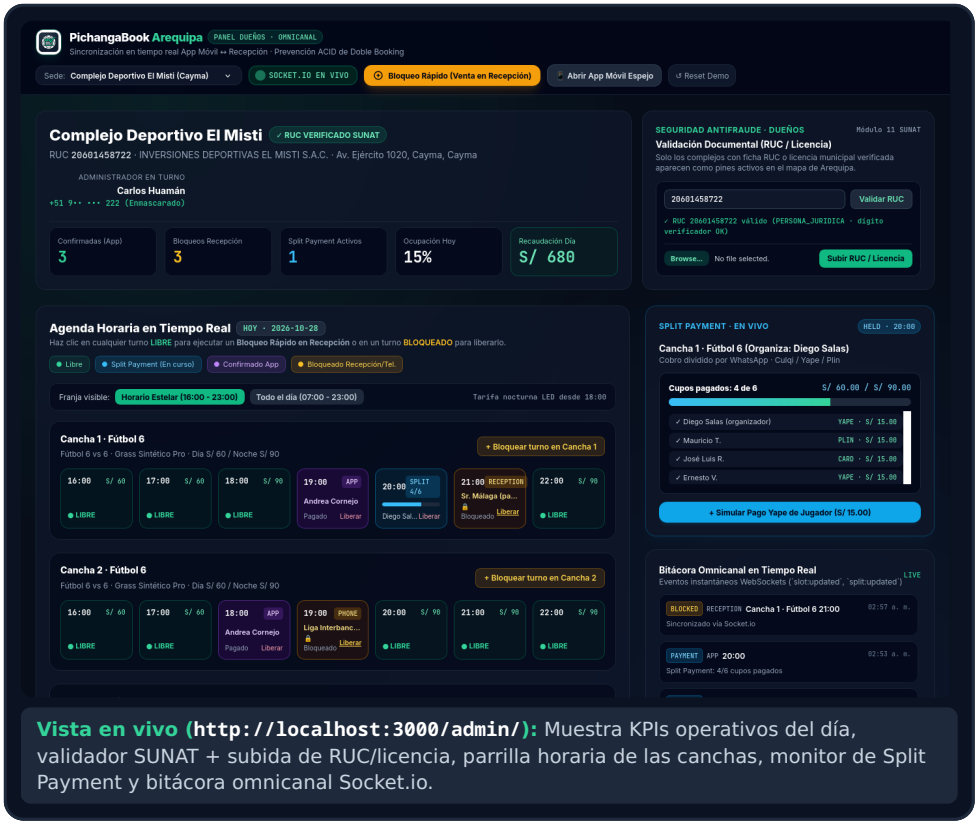
Resultados reales al ejecutar `npm run test:realtime` y `npm run test:concurrency` contra el servidor NestJS + PostgreSQL:

```
$ npm run test:realtime && npm run test:concurrency
=== 1. Health & PostgreSQL Check ===
Health: ok | Driver: pglite (PostgreSQL 18.6 WASM) | Fecha Lima: 2026-10-28
=== 2. Conectando cliente WebSocket (Socket.io) ===
Socket conectado id = jQhBP2UhWttmgntHAAAA
=== 3. Bloqueo Rápido en Recepción (17:00) y prueba de Doble Booking ===
Bloqueo recepción HTTP 201 -> BLOCKED 17:00
Intento doble-booking HTTP 409 -> SLOT_TAKEN Este turno acaba de ser tomado
=== 4. Split Payment (4/6 -> 5/6) con Webhook Culqi HMAC-SHA256 ===
Split actualizado: Cupos pagados: 5 de 6
=== 5. Bolsa de Suplentes + Enmascaramiento Antifraude en Chat/VoIP ===
Proxy VoIP: +51 (054) 6416-809 | Mensaje: Hola crack, mi celular es [número
oculto 🔒]
=== Prueba de 8 reservas paralelas en el mismo milisegundo ===
Códigos HTTP recibidos: 201, 409, 409, 409, 409, 409, 409, 409
✓ Garantía ACID anti doble-booking verificada con éxito.
```

MÓDULO 1 & 6 · FRONTEND WEB PARA DUEÑOS DE COMPLEJOS DEPORTIVOS (/WEB-ADMIN-TAILWIND/INDEX.HTML)

Panel Omnicanal del Dueño, Agenda en Vivo y Validación Documental (RUC / Licencia)

HTML5 + Tailwind CSS + Socket.io



Vista en vivo (`http://localhost:3000/admin/`): Muestra KPIs operativos del día, validador SUNAT + subida de RUC/licencia, parrilla horaria de las canchas, monitor de Split Payment y bitácora omnicanal Socket.io.

Tecnologías Frontend Utilizadas en esta Vista

HTML5 Semántico · Tailwind CSS 3 (Dark Slate & Emerald) · Socket.io Client 4.8 · Fetch API + FormData Multipart

- **Parrilla Horaria Reactiva:** Renderiza dinámicamente cada turno (`LIBRE` , `SPLIT 4/6` , `APP` , `RECEPTION` , `PHONE`) con animación `@keyframes flashPulse` cuando llega un evento WebSocket.
- **Validación Documental en Recepción:** Permite validar el RUC en línea y adjuntar el PDF/imagen de la ficha RUC o licencia municipal (`input type="file"`).

Arquitectura Backend (NestJS + TypeScript) que Alimenta esta Vista

GET /api/owner/complexes/:id/dashboard · GET /api/ruc/:ruc/validate · POST /api/complexes/:id/verification-document

- **`BookingsService.ownerDashboard()` :** Consolida en paralelo los datos del complejo, el estado de verificación documental, la grilla de todas sus canchas, los 5 KPIs financieros y los últimos 20 eventos de `booking_events` .
- **`isValidRuc()` (`src/common/utils/ruc.util.ts`):** Verifica prefijos (`10` , `15` , `17` , `20`) y calcula el **dígito verificador oficial de SUNAT (módulo 11)** con pesos `[5,4,3,2,7,6,5,4,3,2]` .
- **`ComplexesService.uploadVerificationDocument()` :** Procesa el archivo con `Multer FileInterceptor` , lo almacena de forma segura (`MediaService` / S3) y emite el evento `complex:verification` por WebSockets.

Extracto real: backend-nestjs/src/common/utils/ruc.util.ts

```
const WEIGHTS = [5, 4, 3, 2, 7, 6, 5, 4, 3, 2];
export function isValidRuc(ruc: string): boolean {
  if (!/^(10|15|17|20)\d{9}$/.test(ruc)) return false;
  const sum = WEIGHTS.reduce((acc, w, i) => acc + w * Number(ruc[i]), 0);
  let check = 11 - (sum % 11);
  if (check === 10) check = 0; else if (check === 11) check = 1;
  return check === Number(ruc[10]);
}
```

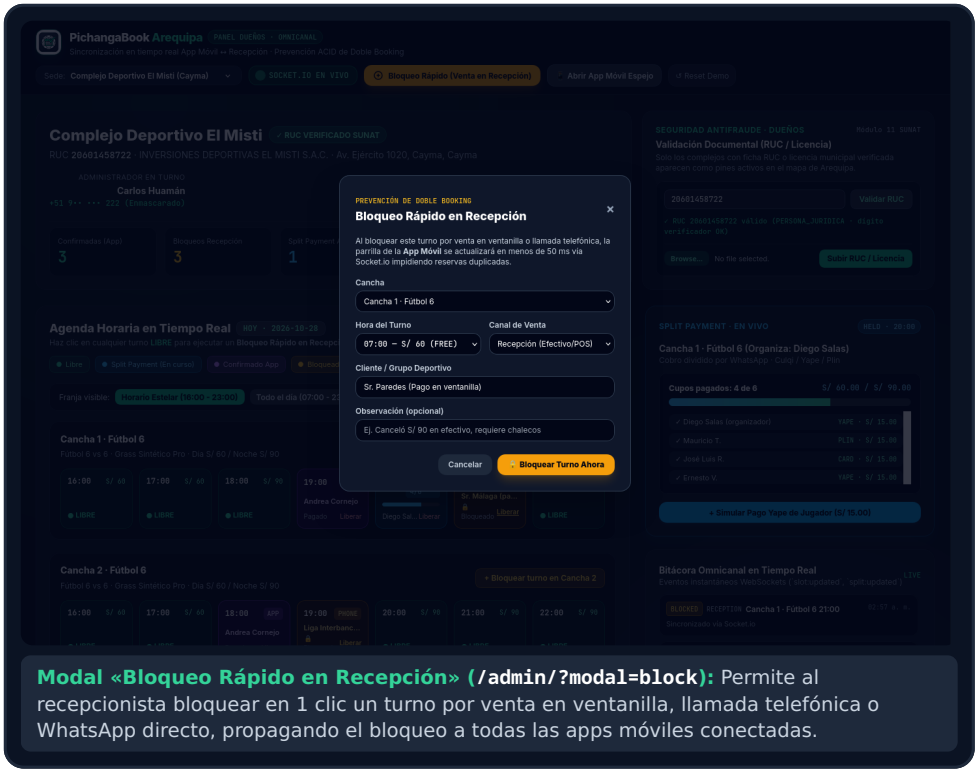
Modelo de Datos PostgreSQL (`schema.sql`)

Tabla: complexes · ENUM: verification_status · CHECK (ruc ~ '^(10|15|17|20)[0-9]{9}$')

- Los complejos en estado `PENDING` o `IN_REVIEW` (ej. *El Misti · Sede Sachaca*) quedan ocultos del mapa público hasta pasar a `VERIFIED` , erradicando estafas con canchas inexistentes.

Bloqueo Rápido en Recepción (Venta Presencial / Teléfono) y Garantía Anti Doble-Booking

Socket.io + PostgreSQL GiST



Modal «Bloqueo Rápido en Recepción» (`/admin/?modal=block`): Permite al recepcionista bloquear en 1 clic un turno por venta en ventanilla, llamada telefónica o WhatsApp directo, propagando el bloqueo a todas las apps móviles conectadas.

Tecnologías Frontend & Comunicación Bidireccional

Modal Interactivo Tailwind CSS · Socket.io Room: complex:<id> · POST /api/bookings/reception-block · WS Event: slot:block & slot:updated

- Al confirmar el bloqueo en recepción, el cliente invoca `POST /api/bookings/reception-block` (o emite `slot:block` por WebSocket) e inmediatamente el servidor notifica `slot:updated` a todos los teléfonos que están viendo esa cancha.

Implementación del `BookingsGateway` (NestJS + Socket.io)

Extracto real: backend-nestjs/src/bookings/bookings.gateway.ts

```
@WebSocketGateway({ cors: { origin: '*', credentials: true }, transports: ['websocket',
'polling'] })
export class BookingsGateway implements OnGatewayConnection, OnModuleInit {
  @WebSocketServer() server: Server;

  onModuleInit(): void {
    this.sub = this.bus.stream$.subscribe(({ rooms, event, payload }) => {
      for (const room of rooms) this.server.to(room).emit(event, payload);
      this.server.to('demo:global').emit(event, payload);
    });
  }

  @SubscribeMessage('slot:block')
  async onSlotBlock(@MessageBody() dto: ReceptionBlockDto) {
    const slot = await this.bookings.receptionBlock(dto);
    return { ok: true, slot };
  }
}
```

Garantía ACID Anti Doble-Booking en PostgreSQL (`btree_gist`)

CREATE EXTENSION btree_gist · EXCLUDE USING gist · SQLSTATE 23P01 → HTTP 409 Conflict

Extracto real: backend-nestjs/schema.sql (Líneas 130-133)

```
-- Garantía física en el motor PostgreSQL contra solapamiento horario en la misma
cancha:
CONSTRAINT bookings_no_overlap EXCLUDE USING gist (
    court_id WITH =,
    tstzrange(starts_at, ends_at, '[)') WITH &&
) WHERE (status IN ('HELD', 'PENDING_PAYMENT', 'CONFIRMED', 'BLOCKED'))
```

Mapa Interactivo de Complejos Sintéticos y Filtro por Distritos de Arequipa

Flutter (Dart) + Google Maps API



Vista Móvil (`/mobile/?tab=map`): Pines con tarifa por hora en Arequipa, chips de distritos clave y cálculo de distancia en km.

Tecnologías Frontend Móvil Utilizadas en esta Vista

Flutter 3 (Dart) · google_maps_flutter ^2.9.0 · geolocator ^13.0.1 · Espejo Web: Leaflet + Carto Dark Tiles

- **Código Nativo Flutter (`lib/screens/arequipa_map_screen.dart`):** Utiliza el widget `GoogleMap` centrado en las coordenadas de Arequipa (`LatLng(-16.3880, -71.5430)`), renderiza `Set<Marker>` para cada complejo verificado y filtra con `ChoiceChip` por distrito (**Cayma, Yanahuara, Cerro Colorado, J.L. Bustamante y Rivero, Sachaca**).
- **Espejo Web Ejecutable (`demo-app-mobile.html`):** Replica el árbol de widgets de Flutter en el navegador conectado en vivo a `GET /api/districts` y `GET /api/complexes`.

Servicios Backend e Integración con Google Maps Platform

GET /api/districts · GET /api/complexes?district=CAYMA&lat=-16.3989&lng=-71.5350 · Google Geocoding & Distance Matrix API

- **`MapsService` (`src/integrations/maps.service.ts`):** Integra `maps.googleapis.com/maps/api/geocode/json` (restringido a `country:PE|administrative_area:Arequipa`) y `distancematrix/json`, con cálculo geométrico de respaldo mediante la fórmula esférica de **Haversine** (`haversineKm`).
- **`ComplexesService.list()`:** Filtra exclusivamente complejos con `verification_status = 'VERIFIED'`, calcula la distancia en km desde la ubicación del jugador y ordena por cercanía o calificación promedio.

Extracto real: mobile-flutter/lib/screens/arequipa_map_screen.dart

```
Set<Marker> _buildMarkers() {
  return _complexes.map((raw) {
    final c = raw as Map<String, dynamic>;
    return Marker(
      markerId: MarkerId(c['id'] as String),
      position: LatLng((c['lat'] as num).toDouble(), (c['lng'] as num).toDouble()),
      infoWindow: InfoWindow(
        title: c['name'] as String,
        snippet: '${c['districtName']} · ★ ${c['rating']} · RUC Verificado',
        onTap: () => widget.onSelectComplex(c),
      ),
    );
  }).toSet();
}
```

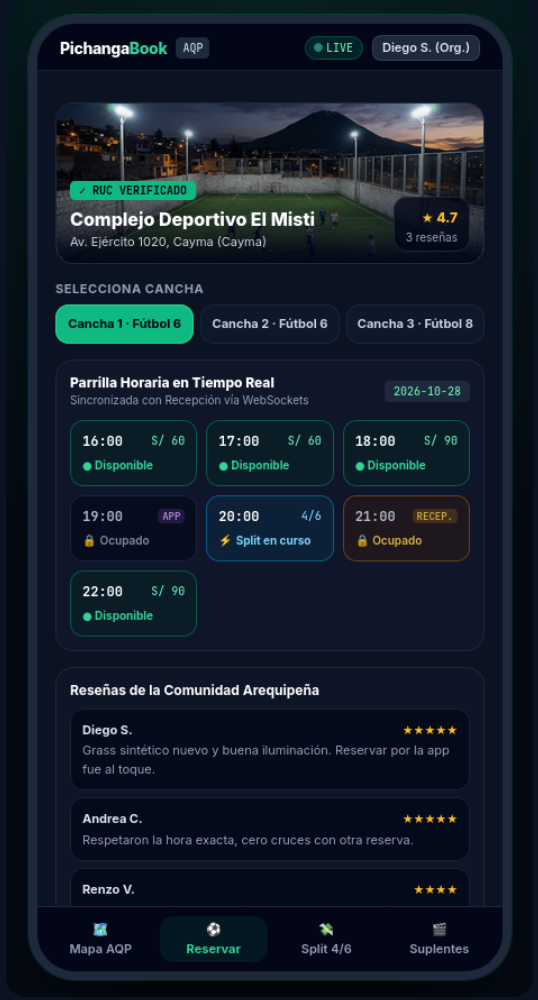
Tablas y Vistas PostgreSQL Involucradas

Tabla: districts (10 distritos AQP) · Tabla: complexes (lat/lng NUMERIC(9,6)) · Vista: v_complex_ratings

- Restricción geográfica en BD para coordenadas de Arequipa: `CHECK (lat BETWEEN -17.0 AND -16.0)` y `CHECK (lng BETWEEN -72.0 AND -71.0)`.

Parrilla Horaria Interactiva del Jugador Sincronizada con Recepción y Comunidad

Flutter + Socket.io + PostgreSQL



Vista Móvil (`/mobile/?tab=courts`): Muestra exactamente los mismos estados que el Panel del Dueño (`19:00 APP`, `20:00 Split 4/6`, `21:00 RECEP.`).

Tecnologías Frontend Móvil Utilizadas en esta Vista

Flutter CourtDetailScreen (GridView.builder) · socket_io_client ^3.0.2 · Modal de Reserva Híbrida (Seña 50% vs Split 6 cupos)

- **Sincronización Instantánea:** El servicio `RealtimeApiService` escucha el evento `slot:updated`. Cuando el dueño bloquea un turno en recepción, la celda correspondiente cambia de `● Disponible` a `🔒 Ocupado (RECEP.)` sin intervención del usuario.
- **Tarifación Diferenciada Día / Noche:** Muestra automáticamente la tarifa diurna (`S/ 60` hasta las 17:00) y la tarifa nocturna con iluminación LED (`S/ 90` desde las 18:00).

Cálculo de Disponibilidad Horaria en el Backend (`AvailabilityService`)

GET /api/complexes/:id · GET /api/courts/:id/availability?date=YYYY-MM-DD · POST /api/complexes/:id/reviews

- **`AvailabilityService.gridForCourts()` (`src/complexes/availability.service.ts`):** Cruza el rango horario de apertura de cada cancha (`open_hour` a `close_hour`) en zona horaria de Lima (`UTC-5`) con las reservas activas (`HELD`, `CONFIRMED`, `BLOCKED`) y el avance de `split_payments`.

Extracto real: backend-nestjs/src/complexes/availability.service.ts

```
SELECT b.id, b.court_id AS "courtId", b.status, b.channel, b.starts_at AS "startsAt",
       b.customer_name AS "customerName", u.full_name AS "organizerName",
       sp.paid_slots AS "paidSlots", sp.total_slots AS "totalSlots", sp.mode::text AS mode
  FROM bookings b
  LEFT JOIN users u ON u.id = b.organizer_id
  LEFT JOIN split_payments sp ON sp.booking_id = b.id
 WHERE b.court_id = ANY($1::uuid[]) AND b.starts_at >= $2 AND b.starts_at < $3
   AND b.status IN ('HELD','PENDING_PAYMENT','CONFIRMED','BLOCKED')
```

Reseñas Verificadas y Prevención de Manipulación

Tabla: courts · Tabla: bookings · Tabla: reviews · UNIQUE (complex_id, user_id)

- La restricción `UNIQUE (complex_id, user_id)` en `reviews` garantiza que cada jugador verificado solo pueda emitir una calificación por complejo deportivo, alimentando la vista agregada `v_complex_ratings`.

Split Payment en Vivo («Cupos pagados: 4 de 6»), Link para WhatsApp y Webhook Culqi

Culqi / Yape / Plin + WhatsApp



Vista Móvil (`/mobile/?tab=split`): Barra segmentada en vivo (4 de 6 cupos pagados), link para WhatsApp y abono con Yape / Plin / Tarjeta.

Tecnologías Frontend Móvil Utilizadas en esta Vista

Flutter SplitPaymentScreen · LinearProgressIndicator / Barra 6 Segmentos · url_launcher (Deep-link wa.me) · WS Event: split:updated

- **Generación de Link para WhatsApp:** Crea un enlace único (`/mobile/?split=pk4de6demo`) pre-formateado con el nombre de la cancha, fecha, hora y cuota individual (`S/ 15.00`) para enviarlo al grupo de WhatsApp del equipo.
- **Actualización Reactiva en Ambos Frentes:** Cada vez que un integrante abona su cuota, tanto la App Móvil como el Panel del Dueño actualizan la barra de progreso (`4/6 → 5/6 → 6/6 CONFIRMADO`).

Transacción ACID con Bloqueo Pesimista (`FOR UPDATE`) y Firma Culqi

GET /api/split-payments/:token · POST /api/split-payments/:token/pay · POST /api/payments/webhooks/culqi · Culqi API v2 + HMAC-SHA256

- **`CulqiService` (`src/integrations/culqi.service.ts`):** Procesa cargos en soles (`PEN`) para tarjeta, Yape y Plin, y verifica la autenticidad del webhook mediante `timingSafeEqual` sobre la firma `HMAC-SHA256` .
- **`BookingsService.recordVerifiedShare()` :** Bloquea la fila con `SELECT ... FROM split_payments WHERE id = $1 FOR UPDATE` , inserta el abono con `ON CONFLICT (provider_charge_id) DO NOTHING` (idempotencia) y, al llegar a `paid_slots = total_slots` , cambia automáticamente la reserva de `HELD` a `CONFIRMED` .

Extracto real: backend-nestjs/src/bookings/bookings.service.ts

```
const [locked] = await tx.query(
  `SELECT id, share_token AS "shareToken", paid_slots AS "paidSlots", total_slots AS "totalSlots"
     FROM split_payments WHERE id = $1 FOR UPDATE`, [params.splitId]);
await tx.query(
  `INSERT INTO payment_shares (split_payment_id, payer_name, method, provider, provider_charge_id, amount_cents)
   VALUES ($1, $2, $3::payment_method, 'CULQI', $4, $5) ON CONFLICT (provider_charge_id) DO NOTHING`, ...);
if (newPaid >= locked.totalSlots) {
  await tx.query(`UPDATE bookings SET status = 'CONFIRMED', hold_expires_at = NULL WHERE id = $1`,
[params.bookingId]);
}
```

Integridad Financiera en PostgreSQL

Tabla: split_payments · CHECK (paid_slots BETWEEN 0 AND total_slots)

Tabla: payment_shares · provider_charge_id UNIQUE

MÓDULO 5 · BOLSA DE SUPLENTES VERIFICADOS — CORE DEL PIVOTE

Reclutamiento de Jugadores de Emergencia con Puntualidad y Clips de Video (15 a 30 s)

Flutter VideoPlayer + Cloudinary / S3



Vista Móvil (`/mobile/?tab=subs`): Tarjetas con posición, puntualidad (`99.1%` / `98.5%`), teléfono enmascarado y reproductor de video de 15–30s.

Tecnologías Frontend Móvil y Streaming Ligero

Flutter SubstitutesPoolScreen · video_player ^2.9.2 (VideoPlayerController) · HTML5 <video preload="metadata" playsinline> · Clips H.264 480p + faststart

- **Validación Visual del Nivel Técnico:** Cada tarjeta incorpora un minirreproductor de video con clips cortos de **15 a 30 segundos** (`arquero.mp4` 22s, `delantero.mp4` 18s, `defensa.mp4` 25s, `mediocampo.mp4` 28s) junto al indicador histórico de puntualidad (`punctuality_score`) y partidos jugados.

Ingesta Firmada y Optimización Multimedia en Cloudinary / AWS S3

GET /api/substitutes?position=ARQUERO · POST /api/substitutes/clip-upload-signature · Cloudinary Signed Upload (SHA-1)

- **`MediaService.signClipUpload()` (`src/integrations/media.service.ts`):** Genera una firma criptográfica `SHA-1` para que la app móvil suba el video directamente a Cloudinary/S3 sin saturar el backend, aplicando transformación automática `so_0,eo_30,q_auto:eco,w_480,vc_auto/mp4` (recorte a 30s y compresión adaptativa).

Extracto real: backend-nestjs/src/integrations/media.service.ts

```
signClipUpload(userId: string) {
  const timestamp = Math.floor(Date.now() / 1000);
  const params = {
    folder: 'pichangabook/clips',
    public_id: `sub_${userId}_${timestamp}`,
    eager: `so_0,eo_${CLIP_MAX_SECONDS},q_auto:eco,w_480,vc_auto/mp4`,
    timestamp,
  };
  const toSign = Object.keys(params).sort().map((k) => `${k}=${params[k]}`).join('&');
  const signature = createHash('sha1').update(toSign + (apiSecret || 'demo-secret')).digest('hex');
  return { uploadUrl: `https://api.cloudinary.com/v1_1/${cloudName}/video/upload`, signature, ...params };
}
```

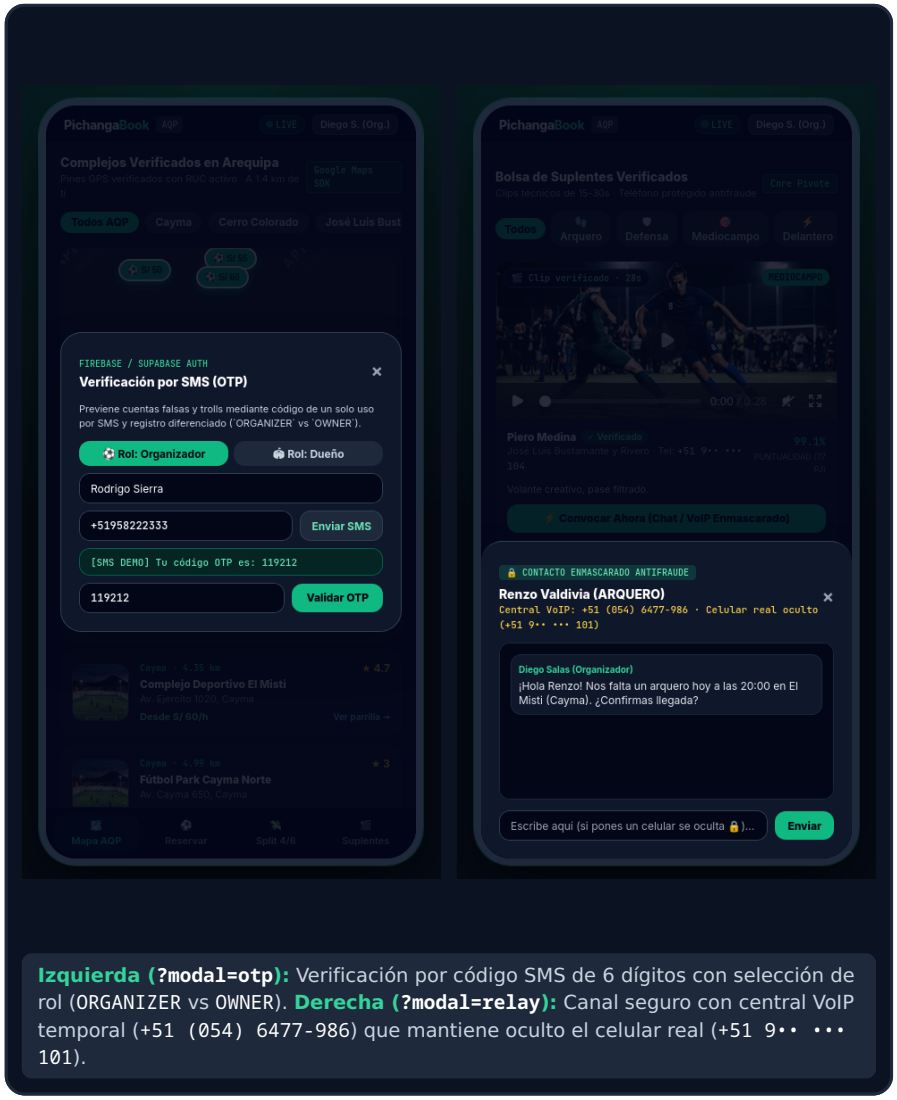
Esquema e Índice de Búsqueda Rápida en PostgreSQL

Tabla: substitute_profiles · CHECK (clip_duration_s BETWEEN 15 AND 30) · INDEX idx_substitutes_lookup (position, is_available, punctuality_score DESC)

- El índice compuesto `idx_substitutes_lookup` permite encontrar en milisegundos al suplente disponible más puntual para la posición requerida a última hora.

Registro Diferenciado con OTP SMS y Enmascaramiento Telefónico (Chat / Central VoIP)

Firebase Auth + Proxy VoIP



Izquierda (`?modal=otp`): Verificación por código SMS de 6 dígitos con selección de rol (`ORGANIZER` vs `OWNER`). **Derecha (`?modal=relay`):** Canal seguro con central VoIP temporal (`+51 (054) 6477-986`) que mantiene oculto el celular real (`+51 9•• ••• 101`).

1. Autenticación Segura OTP por SMS y Registro por Rol

Firebase Auth / Supabase Auth · POST /api/auth/otp/request & /verify · POST /api/auth/firebase · @nestjs/jwt + AuthGuard (@Roles)

- **Mitigación de Trolls y Reservas Falsas:** Ningún usuario puede reservar ni publicar canchas sin validar su línea móvil mediante un código de un solo uso (OTP de 6 dígitos con expiración de 5 min y límite de 5 intentos en `OtpService`).
- **Control de Acceso Basado en Roles (RBAC):** El guard global `AuthGuard` verifica el token JWT firmado y restringe las rutas administrativas exclusivamente a `@Roles('OWNER', 'ADMIN')`.

2. Protección contra Extorsión: Enmascaramiento E.164 y Redacción Automática

POST /api/substitutes/:id/invite · POST /api/relays/:id/messages · Columna Generada: phone_masked

- **Enmascaramiento en Base de Datos:** La API jamás serializa `phone_e164`; solo expone la columna calculada `phone_masked` (`+51 9•• ••• 101`).
- **Central Proxy VoIP y Sanitización de Mensajes:** Al convocar a un suplente se asigna un número virtual temporal (`+51 (054) 64XX-XXX`) con vigencia de 4 horas y el filtro `redactContactInfo()` bloquea cualquier intento de filtrar números celulares o correos en el chat.

Extracto real: backend-nestjs/schema.sql & src/common/utils/phone.util.ts

```
-- En schema.sql (Columna generada almacenada en PostgreSQL):
phone_masked VARCHAR(24) GENERATED ALWAYS AS (
  left(phone_e164, 3) || ' ' || substr(phone_e164, 4, 1) || '•• ••• ' || right(phone_e164, 3)
) STORED

// En src/common/utils/phone.util.ts (Filtro antifraude en mensajes del chat interno):
const PHONE_IN_TEXT = /(\+?\s?51[\s.-]?)?\b9(?:[\s.-]?\d){8}\b|\b\d(?:[\s.-]?\d){8,13}\b/g;
export function redactContactInfo(text: string): { body: string; redacted: boolean } {
  let redacted = false;
  const body = text.replace(PHONE_IN_TEXT, () => { redacted = true; return '[número oculto
🔒]'; });
  return { body, redacted };
}
```